

MARKSHEET ANALYTICS APP

PRD + Full Project Structure + Analytics Design

Flutter • FastAPI • Google Document AI • Supabase

College Edition — CGPA / SGPA

1. CORE CONCEPT — PROJECT-BASED MODEL

1.1 What Is a 'Project'?

A Project is a single exam event for one class — for example: 'CSE Sem 4 Mid-Term 2024'. Each project is fully isolated and has its own:

- Student marksheet uploads (PDFs)
- Extracted data + Excel sheet
- Analytics dashboard (toppers, CGPA/SGPA, pass/fail)
- Shareable link (optional)

This is exactly like Google Forms — each form is one project, each form has its own response sheet and analytics. Creating a new project starts fresh with zero data from other projects.

1.2 Project Lifecycle

Step	Action	Who
1	Teacher creates a new Project (name, class, semester, exam type)	Teacher
2	Uploads marksheet PDFs (single or class batch)	Teacher
3	App extracts + validates student data via OCR pipeline	System
4	Teacher reviews flagged rows and corrects them	Teacher
5	System generates Excel sheet for this project	System
6	Analytics dashboard auto-populates for this project	System
7	Teacher downloads Excel or views dashboard anytime	Teacher

1.3 Project Isolation Rule

Project A and Project B NEVER share data. Each project has:

- Its own Supabase storage bucket path: `projects/{project_id}/files/`
- Its own database rows in `student_rows` table filtered by `project_id`
- Its own generated Excel file: `projects/{project_id}/export.xlsx`
- Its own analytics computed from only that project's student data

2. PRODUCT REQUIREMENTS DOCUMENT

2.1 Problem Statement

- College teachers waste hours manually entering marks from paper marksheets into Excel
- No affordable tool exists for Indian colleges to go from scanned marksheet → CGPA/SGPA analytics
- Teachers have no quick way to see class performance, toppers, and fail counts after exams

2.2 Goals

Goal	Description
Scan to Excel	Teacher uploads marksheet PDFs → app auto-generates structured Excel
Project isolation	Each class/exam is its own project with separate data and analytics
CGPA / SGPA calculation	System computes GPA per student using grade point scale
Analytics dashboard	Per-project: topper list, GPA distribution, pass/fail breakdown
Human-in-loop review	Flagged rows shown to teacher for quick correction before finalizing
Download + share	Export Excel and share dashboard link with HOD or students

2.3 Feature List with Priority

Feature	P	Description
Create / manage Projects	P0	Each project = one class + one exam. CRUD for projects.
PDF upload	P0	Upload single or batch marksheet PDFs per project
OCR + Data Extraction	P0	Google Document AI → extract roll no, name, subjects, marks
CGPA / SGPA calculator	P0	Convert marks to grade points using configurable scale
Validation layer	P0	Flag total mismatch, CGPA > 10, missing fields
Review + Edit UI	P0	Show only flagged rows; teacher corrects inline
Excel Generator	P0	Roll Name Sub1 Sub2 Total SGPA CGPA Grade
Analytics Dashboard	P0	Per project: toppers, pass/fail, GPA distribution
Project History	P1	List all past projects with quick re-open
Template Manager	P1	Define column layout once per marksheet format
Multi-student PDF split	P1	Auto-detect and split class-level PDFs by student
Share dashboard link	P2	Public read-only link for HOD / students

2.4 Tech Stack

Layer	Technology	Why
Mobile App	Flutter (Dart)	Cross-platform iOS + Android

Layer	Technology	Why
State Management	Riverpod	Provider pattern, clean separation
Navigation	GoRouter	Declarative routing
HTTP Client	Dio	Interceptors for auth tokens
Backend	FastAPI (Python)	Async, fast, ML-friendly
OCR	Google Document AI	Free tier, best for Indian college formats
Database	Supabase (PostgreSQL)	Auth + DB + Storage in one
File Storage	Supabase Storage	Store PDFs + Excel per project
Excel Output	openpyxl + pandas	Reliable, styled Excel generation
PDF Splitting	PyMuPDF (fitz)	Fast page-level PDF manipulation
Deployment	Railway.app	Simple deploy, free tier
Analytics Compute	Python (backend)	All GPA/stats computed server-side

3. ANALYTICS DASHBOARD — FULL DESIGN

3.1 Dashboard Cards (per Project)

Card	Data Shown	How Computed
Class Overview	Total students, Appeared, Absent	Count of rows in project
Pass / Fail Count	Passed: N Failed: N Pass %	Failed = any subject marks < passing_marks
Topper Board	Rank 1, 2, 3 — Name + SGPA/CGPA	Sorted descending by SGPA
GPA Distribution	Buckets: 9-10 / 8-9 / 7-8 / 6-7 / <6	Count of students per SGPA range
Subject-wise Avg	Each subject average mark	mean(marks) per subject
Grade Distribution	O / A+ / A / B+ / B / C / F count	Grade assigned by grade point scale
Above / Below Average	Students above class avg vs below	Compare each SGPA to class mean SGPA
Highest / Lowest	Max SGPA, Min SGPA, Class Average SGPA	Simple min/max/mean computation

3.2 CGPA / SGPA Calculation Logic

Default 10-point scale (configurable per project):

Marks Range	Grade	Grade Points
90 - 100	O (Outstanding)	10
80 - 89	A+ (Excellent)	9
70 - 79	A (Very Good)	8
60 - 69	B+ (Good)	7
50 - 59	B (Above Average)	6
40 - 49	C (Satisfactory)	5
Below 40	F (Fail)	0

SGPA Formula (for one semester):

$$SGPA = \sum(\text{grade_points}[i] \times \text{credits}[i]) / \sum(\text{credits}[i])$$

CGPA Formula (across multiple semesters — only if multi-sem project):

$$CGPA = \sum(SGPA[\text{sem}] \times \text{total_credits}[\text{sem}]) / \sum(\text{total_credits}[\text{sem}])$$

If project is single-exam (mid-term, end-term), SGPA is the primary metric. CGPA shown only when teacher links multiple semester projects.

3.3 Topper / Rank List Logic

```
# Ranking rules:
# 1. Sort by SGPA descending
```

```
# 2. Tie-break: higher total marks wins
# 3. Tie-break 2: alphabetical by name
# 4. Students with F grade shown at bottom regardless of SGPA
```

Rank list columns in app: Rank | Roll No | Name | SGPA | Grade | Status (Pass/Fail)

3.4 Pass / Fail Rules

- Student PASSES if: marks $\geq$ passing_marks in ALL subjects
- Student FAILS if: marks $<$ passing_marks in ANY subject
- Default passing marks = 40 (configurable per project)
- ATKT (Allowed To Keep Term): fails in 1-2 subjects — shown as separate status
- Analytics shows: Pass count / ATKT count / Fail count / Absent count

3.5 Above / Below Average Widget

For each student, compare their SGPA against the class average SGPA:

```
above_avg = [s for s in students if s.sgpa >= class_avg_sgpa]
below_avg = [s for s in students if s.sgpa < class_avg_sgpa]
```

Displayed as a horizontal bar with count + percentage. Clicking a bar filters the student list.

4. FLUTTER APP — COMPLETE FOLDER STRUCTURE

4.1 Top Level

lib/	
main.dart	← ProviderScope, Supabase.initialize
app.dart	← MaterialApp.router, theme
router.dart	← GoRouter: all routes defined here
core/	← App-wide config, network, utils
features/	← One folder per feature
shared/	← Reusable widgets + providers

4.2 Core Layer

core/	
constants/	
app_colors.dart	← Primary, accent, error, neutral palette
app_text_styles.dart	← h1/h2/body/caption text styles
app_strings.dart	← All UI strings (no hardcoding in widgets)
api_endpoints.dart	← All backend URL constants
errors/	
app_exception.dart	← Custom exception types
failure.dart	← Failure sealed class (network/server/parse)
network/	
dio_client.dart	← Dio setup, baseUrl, timeout
auth_interceptor.dart	← Inject JWT, handle 401 auto-refresh
storage/	
local_storage.dart	← SharedPreferences wrapper
secure_storage.dart	← flutter_secure_storage for JWT
utils/	
file_utils.dart	← File size, extension, mime type helpers
gpa_utils.dart	← Grade point conversion functions
validators.dart	← Form field validation functions
date_utils.dart	← Format date/time strings

4.3 Features — Auth

features/auth/	
data/	
auth_repository.dart	← Supabase login/signup/logout calls
auth_dto.dart	← LoginRequest, SignupRequest, TokenResponse
domain/	
teacher_model.dart	← Teacher(id, email, name, college, department)
presentation/	
screens/	
login_screen.dart	
signup_screen.dart	
widgets/	
auth_input_field.dart	← Styled text field for auth forms
auth_submit_button.dart	← Handles loading state
college_name_field.dart	← College + department inputs
providers/	
auth_provider.dart	← StateNotifier<AuthState>

auth_state.dart ← Sealed: initial / loading / authenticated / error

4.4 Features — Projects (CORE)

```
features/projects/
  data/
    project_repository.dart ← CRUD API calls for projects
    project_dto.dart       ← CreateProjectRequest, ProjectResponse
  domain/
    project_model.dart     ← Project(id, name, class, semester, exam_type,
                             passing_marks, credits_map, created_at)
  presentation/
    screens/
      projects_list_screen.dart ← Home screen: list of all teacher's projects
      create_project_screen.dart ← Form: name, class, semester, exam type, passing marks
      project_detail_screen.dart ← Tabs: Upload | Review | Analytics | Export
    widgets/
      project_card.dart      ← Card: project name, date, student count, status chip
      create_project_fab.dart ← FAB to open create project bottom sheet
      project_status_chip.dart ← Processing / Ready / Reviewed chip
      project_detail_tabs.dart ← TabBar: Upload / Review / Analytics / Export
      empty_projects_view.dart ← Illustration + 'Create your first project' CTA
    providers/
      projects_provider.dart ← StateNotifier<ProjectsState>: list of projects
      project_detail_provider.dart ← StateNotifier for one open project
      projects_state.dart     ← Sealed: loading / loaded / error
```

4.5 Features — Upload

```
features/upload/
  data/
    upload_repository.dart ← POST /upload with project_id
    upload_dto.dart        ← UploadResponse, JobStatus
  domain/
    upload_job_model.dart  ← Job(id, project_id, status, progress, created_at)
  presentation/
    screens/
      upload_screen.dart   ← Upload tab inside project_detail_screen
    widgets/
      upload_drop_zone.dart ← Tap or drag to pick files
      file_list_tile.dart   ← Shows each selected file: name, size, remove btn
      upload_progress_card.dart ← Per-file progress bar + status
      template_selector_row.dart ← Pick marksheet template for this upload
      job_status_banner.dart ← Processing / Done / Error banner
    providers/
      upload_provider.dart ← StateNotifier<UploadState>
      upload_state.dart    ← Sealed: idle / picking / uploading / processing / done
```

4.6 Features — Review

```
features/review/
  data/
    review_repository.dart ← GET extracted rows, PATCH row correction
    review_dto.dart        ← StudentRowDto, CorrectionRequest
  domain/
```



```

student_row_model.dart      ← id, roll_no, name, subject_marks, sgpa, errors[]
validation_error_model.dart ← type (enum), message, field_name
presentation/
screens/
  review_screen.dart        ← Review tab: All rows / Flagged only toggle
widgets/
  student_row_card.dart     ← Full row: roll, name, marks, SGPA – red if flagged
  flagged_row_editor.dart   ← Expanded edit panel for flagged row
  inline_mark_field.dart    ← Tap cell → edit single mark
  validation_error_chip.dart ← Shows error type badge (TOTAL_MISMATCH etc.)
  review_summary_bar.dart   ← 'N rows flagged, N confirmed' progress bar
  confirm_all_button.dart   ← Bulk confirm all reviewed rows
providers/
  review_provider.dart      ← StateNotifier<ReviewState>
  review_state.dart         ← loading / loaded(rows, flaggedOnly) / saving / done

```

4.7 Features — Analytics Dashboard

```

features/analytics/
data/
  analytics_repository.dart ← GET /analytics/{project_id}
  analytics_dto.dart        ← AnalyticsSummaryDto, TopperDto, DistributionDto
domain/
  analytics_model.dart      ← Full analytics model (all cards)
  topper_model.dart         ← Rank, roll_no, name, sgpa, grade
  gpa_bucket_model.dart     ← Range label + student count
  subject_avg_model.dart    ← Subject name + average marks
presentation/
screens/
  analytics_screen.dart     ← Analytics tab: scrollable dashboard
widgets/
  overview_summary_card.dart ← Total / Appeared / Pass / Fail counts
  pass_fail_card.dart        ← Pass N | ATKT N | Fail N with color bar
  topper_board_card.dart     ← Rank 1/2/3 with trophy icons + SGPA
  full_rank_list_tile.dart   ← Scrollable rank list: all students
  gpa_distribution_chart.dart ← Bar chart: SGPA buckets 9-10 / 8-9 etc.
  grade_distribution_card.dart ← O/A+/A/B+/B/C/F count chips
  above_below_avg_card.dart  ← Horizontal bar: above avg vs below avg
  class_stats_card.dart      ← Highest / Lowest / Average SGPA
providers/
  analytics_provider.dart    ← StateNotifier<AnalyticsState>
  analytics_state.dart       ← loading / loaded(analytics) / error

```

4.8 Features — Export

```

features/export/
data/
  export_repository.dart    ← GET /export/{project_id} → download file
presentation/
screens/
  export_screen.dart        ← Export tab: preview + download button
widgets/
  export_preview_table.dart ← Mini table preview of Excel columns
  download_button.dart      ← Download + open in Files app
  share_button.dart         ← Share Excel via WhatsApp / email

```

```

providers/
  export_provider.dart      ← StateNotifier<ExportState>

```

4.9 Features — Template Manager

```

features/template/
  data/
    template_repository.dart
    template_dto.dart      ← TemplateCreate, TemplateResponse
  domain/
    template_model.dart    ← id, name, columns[], credits_map, max_marks_map
    column_definition.dart ← column_type (roll/name/subject/total), label
  presentation/
    screens/
      template_list_screen.dart
      template_editor_screen.dart
    widgets/
      column_tile.dart      ← Draggable tile per column
      column_type_picker.dart ← Subject / Roll / Name / Total chip selector
      credits_input_row.dart ← Set credit weight per subject
  providers/
    template_provider.dart

```

4.10 Shared Layer

```

shared/
  widgets/
    app_button.dart        ← Primary / secondary / danger + loading state
    app_text_field.dart    ← Consistent input with label + error
    app_loader.dart        ← Full-screen + inline shimmer loaders
    app_error_view.dart    ← Error illustration + retry button
    app_empty_view.dart    ← Empty state with icon + message
    app_snackbar.dart      ← Success / error / info snackbars
    stat_chip.dart         ← Colored chip for stats (Pass: 42)
    section_card.dart      ← White rounded card wrapper for dashboard sections
    confirmation_dialog.dart ← Reusable confirm/cancel dialog
  providers/
    supabase_provider.dart ← Global Supabase client
    dio_provider.dart       ← Global Dio instance
    theme_provider.dart     ← Light/dark theme

```

5. BACKEND — COMPLETE FOLDER STRUCTURE

5.1 Top Level

```
backend/
  main.py                ← FastAPI app, CORS, include routers
  requirements.txt
  .env                  ← SUPABASE_URL, SUPABASE_KEY, GOOGLE_DOC_AI_KEY
  Dockerfile

  app/
    api/                ← Route files
    core/               ← Config, DB, security
    models/             ← SQLAlchemy models
    schemas/            ← Pydantic schemas
    services/           ← All business logic
    utils/              ← Helpers
    tests/
```

5.2 API Routes — app/api/

```
app/api/
  auth.py               ← POST /auth/login /auth/signup /auth/refresh
  projects.py           ← GET/POST/PUT/DELETE /projects /projects/{id}
  upload.py             ← POST /upload (multipart: files + project_id + template_id)
  jobs.py               ← GET /jobs/{id} (poll processing status)
  review.py             ← GET /review/{project_id}
                       ← PATCH /review/{project_id}/row/{row_id}
  analytics.py          ← GET /analytics/{project_id}
  export.py             ← GET /export/{project_id} (streams .xlsx file)
  templates.py          ← CRUD /templates
```

5.3 Models — app/models/

```
app/models/
  teacher.py            ← id, email, name, college, department, created_at
  project.py            ← id, teacher_id, name, class_name, semester,
                       exam_type, passing_marks, credits_json, status
  upload_job.py         ← id, project_id, status, file_count, created_at
  student_row.py        ← id, project_id, job_id, roll_no, name,
                       subject_marks_json, total, sgpa, grade,
                       is_flagged, is_confirmed, errors_json
  template.py           ← id, teacher_id, name, columns_json, credits_json
```

5.4 Schemas — app/schemas/

```
app/schemas/
  auth_schema.py        ← LoginRequest, SignupRequest, TokenResponse
  project_schema.py     ← ProjectCreate, ProjectResponse, ProjectUpdate
  upload_schema.py      ← UploadResponse, JobStatusResponse
  student_schema.py     ← StudentRow, SubjectMark, ValidationErrorSchema
  review_schema.py      ← ReviewResponse, CorrectionRequest
  analytics_schema.py   ← AnalyticsSummary, TopperItem, GpaBucket, SubjectAvg
  template_schema.py    ← TemplateCreate, TemplateResponse, ColumnDefinition
```

5.5 Services — app/services/ (BUSINESS LOGIC)

ocr_service.py

```
# Calls Google Document AI
# Input:  file path (PDF or image)
# Output: list of raw text blocks + detected tables
```

extraction_service.py

```
# Input:  OCR blocks + template column definitions
# Output: list of StudentRow objects
# Logic:
#   - Match columns by header labels from template
#   - For each data row → map to roll_no, name, subject_marks dict
#   - Handle both single-student and multi-student layouts
```

gpa_service.py

```
# Input:  subject_marks dict + credits_map + grade_scale
# Output: sgpa (float), grade (str), grade_points dict

def marks_to_grade_points(marks: int, scale: dict) -> float:
    for range_key, gp in scale.items():
        low, high = range_key
        if low <= marks <= high:
            return gp
    return 0.0 # Below threshold = F

def compute_sgpa(marks_dict, credits_map, scale) -> float:
    total_credits = sum(credits_map.values())
    weighted = sum(
        marks_to_grade_points(marks, scale) * credits_map[subj]
        for subj, marks in marks_dict.items()
    )
    return round(weighted / total_credits, 2)
```

validation_service.py

```
# Rules applied to each StudentRow:
# TOTAL_MISMATCH → total != sum(subject_marks.values())
# INVALID_SGPA   → sgpa > 10 or sgpa < 0
# MISSING_FIELD  → roll_no or name is blank
# MARK_OVERFLOW  → any subject mark > max_marks for that subject
# ZERO_CREDITS   → subject has 0 credits in credits_map
```

analytics_service.py

```
# Input:  all confirmed StudentRow records for a project_id
# Output: AnalyticsSummary object

# Computes:
#   - total / appeared / absent count
#   - pass / atkt / fail / absent counts
```

```
# - rank list (sort by sgpa desc, tie-break by total marks)
# - gpa_buckets: {'9-10': N, '8-9': N, '7-8': N, ...}
# - grade_distribution: {'O': N, 'A+': N, 'A': N, ...}
# - subject_averages: {'Math': 72.4, 'Physics': 68.1, ...}
# - above_avg / below_avg counts vs class mean SGPA
# - highest_sgpa, lowest_sgpa, class_avg_sgpa
```

export_service.py

```
# Generates styled .xlsx using openpyxl
# Sheet 1: Student Data
#   Roll | Name | Sub1 | Sub2 | ... | Total | SGPA | Grade | Status
# Sheet 2: Analytics Summary
#   Class avg, topper, pass/fail counts
# Sheet 3: Rank List
#   Rank | Roll | Name | SGPA | Grade | Pass/Fail
```

pdf_service.py

```
# Splits multi-student PDFs using PyMuPDF
# Detects student boundary by: page break OR roll_no pattern at top of region
```

storage_service.py

```
# All file I/O with Supabase Storage
# Upload:  projects/{project_id}/uploads/{filename}
# Export:  projects/{project_id}/export.xlsx
```

5.6 Full API Map

Method	Endpoint	Returns
POST	/auth/signup	Teacher account + JWT
POST	/auth/login	JWT token
GET	/projects	List of teacher's projects
POST	/projects	New project object
GET	/projects/{id}	Single project detail
PUT	/projects/{id}	Updated project
DELETE	/projects/{id}	204 No Content
POST	/upload	Job id + status
GET	/jobs/{id}	Job status + progress %
GET	/review/{project_id}	All student rows + errors
PATCH	/review/{project_id}/row/{row_id}	Updated row after correction
GET	/analytics/{project_id}	Full analytics summary object
GET	/export/{project_id}	Streams .xlsx file
GET	/templates	Teacher's saved templates
POST	/templates	New template

Method	Endpoint	Returns
PUT	/templates/{id}	Updated template
DELETE	/templates/{id}	204 No Content

6. DATABASE SCHEMA (Supabase / PostgreSQL)

6.1 Tables

```
teachers
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4()
  email       TEXT UNIQUE NOT NULL
  name        TEXT
  college     TEXT
  department  TEXT
  created_at  TIMESTAMPTZ DEFAULT now()

projects
  id          UUID PRIMARY KEY
  teacher_id  UUID REFERENCES teachers(id)
  name        TEXT NOT NULL          -- 'CSE Sem4 Mid-Term 2024'
  class_name  TEXT                   -- 'CSE B'
  semester    INT                     -- 4
  exam_type   TEXT                   -- 'mid-term' | 'end-term'
  passing_marks INT DEFAULT 40
  credits_json JSONB                 -- {'Math': 4, 'Physics': 3}
  grade_scale JSONB                   -- custom grade scale if needed
  status      TEXT DEFAULT 'created' -- created/processing/reviewed/exported
  created_at  TIMESTAMPTZ DEFAULT now()

upload_jobs
  id          UUID PRIMARY KEY
  project_id  UUID REFERENCES projects(id)
  status      TEXT -- queued/processing/done/failed
  file_count  INT
  created_at  TIMESTAMPTZ DEFAULT now()

student_rows
  id          UUID PRIMARY KEY
  project_id  UUID REFERENCES projects(id)
  job_id      UUID REFERENCES upload_jobs(id)
  roll_no     TEXT
  name        TEXT
  subject_marks JSONB -- {'Math': 78, 'Physics': 65}
  total       INT
  sgpa        NUMERIC(4,2)
  grade       TEXT -- O/A+/A/B+/B/C/F
  status      TEXT -- pass/atkt/fail/absent
  is_flagged  BOOLEAN DEFAULT false
  is_confirmed BOOLEAN DEFAULT false
  errors      JSONB -- [{type, message, field}]
  rank        INT -- computed after all rows confirmed

templates
  id          UUID PRIMARY KEY
  teacher_id  UUID REFERENCES teachers(id)
  name        TEXT
```

```
columns      JSONB      -- [{type, label, order}]
credits_map  JSONB
max_marks    JSONB
created_at   TIMESTAMPTZ DEFAULT now()
```


7. DEPENDENCIES

7.1 Flutter pubspec.yaml

Package	Purpose
flutter_riverpod: ^2.5.0	State management — providers, StateNotifier
go_router: ^13.0.0	Declarative navigation
dio: ^5.4.0	HTTP client with interceptors
supabase_flutter: ^2.3.0	Auth + storage
file_picker: ^8.0.0	Pick PDF files from device
image_picker: ^1.0.7	Camera capture option
open_filex: ^4.3.4	Open downloaded Excel on device
path_provider: ^2.1.2	Device file paths
shared_preferences: ^2.2.3	Local non-sensitive settings
flutter_secure_storage: ^9.0.0	Secure JWT storage
freezed: ^2.4.7	Immutable model + state classes
json_serializable: ^6.7.1	JSON to/from Dart models
fl_chart: ^0.68.0	Bar charts for GPA distribution
shimmer: ^3.0.0	Loading skeleton effect

7.2 Backend requirements.txt

Package	Purpose
fastapi	Web framework
uvicorn[standard]	ASGI production server
pydantic[email]	Request/response validation
pydantic-settings	Config from .env
sqlalchemy[asyncio]	Async ORM
asyncpg	Async PostgreSQL driver
alembic	DB migrations
python-jose[cryptography]	JWT token creation + validation
passlib[bcrypt]	Secure password hashing
python-multipart	Multipart file upload parsing
google-cloud-documentai	OCR — Google Document AI
pymupdf	PDF splitting and manipulation
openpyxl	Excel file creation with styling

Package	Purpose
pandas	Data processing for analytics
supabase	Storage + database client
pytest + httpx	Unit + integration testing

8. BUILD ROADMAP

Phase 1 — Core Pipeline (Week 1–2)

- Flutter: Auth screens (login / signup)
- Flutter: Projects list screen + create project form
- Backend: /auth, /projects, /upload, /jobs endpoints
- Backend: OCR service + extraction service (single PDF)
- Backend: gpa_service — SGPA computation
- Flutter: Upload tab with file picker + job status polling
- Backend: export_service basic (Sheet 1 only)
- Flutter: Download + open Excel

Phase 2 — Review + Analytics (Week 3–4)

- Backend: validation_service with all 5 rules
- Flutter: Review screen — flagged row editor
- Backend: analytics_service — all 8 analytics cards
- Flutter: Analytics screen — all widgets
- Backend: export_service Sheet 2 + Sheet 3 (analytics + rank list)
- Flutter: Template manager UI
- Backend: template CRUD

Phase 3 — Polish (Week 5–6)

- Multi-student PDF splitting
- History screen per project with re-download
- All error / empty / loading states on every screen
- Share Excel via WhatsApp / email
- Deploy backend to Railway
- Deploy Supabase production instance